Data Structures and algorithms
Instructor : **Dr.Amin Eskandari**
Head-Ta's : Amir Mohammad Asadjoo, Ali Nikouan
Ta's: Mehdi Heidari, Narges Hosseini, Abolfazl Tadrisi, Arvin Afshari, Amir Arshia Tootoonchi, Fatemeh Feyzi

---

سوال اول:

توابع زیر را از نظر مرتبه رشد از کند ترین تا سریع ترین مرتب کنید.

نکته: بعضی از آنها باهم برابر هستند. مشخص کنید و توضیح دهید چرا برابر هستند

$$n^3, \pi n, 1, 2^n, n\log n, \alpha n, n!, \log n, 1000n, n^2, n$$

سوال دوم:

با توجه به توضیحات هر بخش. مرتبه زمانی آن بخش را مشخص کنید.

الف) چاپ کردن اولین عنصر یک ارایه

ب) الگوریتمی "تمام" عناصر یک آرایه را "یک بار" پیمایش کرده و مجموع آن ها را محاسبه میکند

ج) الگوریتمی هر عنصر آرایه را با تمام عناصر دیگر مقایسه میکند.

د) الگوریتمی در هر مرحله اندازه ی آرایه را نصف میکند و همین کار را روی بخش باقی مانده ادامه میدهد

ه) الگوریتمی در هر مرحله اندازه ی آرایه را نصف میکند و روی" کل" آرایه یک پیمایش کامل انجام میدهد.

سوال سوم:

پیچیدگی زمانی رابطه های بازگشتی زیر را به دست آورید و در هر سوال از روش مناسب استفاده کنید.

$$t(n) = 2t\left(\frac{n}{2}\right) + 1$$

$$t(n) = 9t\left(\frac{n}{3}\right) + n\sqrt{n}$$

$$t(n) = 4t\left(\frac{n}{2}\right) + n^2$$

$$t(n)16t\left(\frac{n}{4}\right) + n^2 \log n$$

$$t(n) = 3t\left(\frac{n}{2}\right) + n^2$$

دو سوال زیر را فقط با روش درخت بازگشتی حل کنید. استفاده از مستر مجاز نیست.

$$t(n) = 2t\left(\frac{n}{2}\right) + n^2$$

$$t(n) = t\left(\frac{n}{2}\right) + t\left(\frac{n}{4}\right) + n$$

برای سوالات زیر از روش های آموزش داده شده و مناسب استفاده کنید.

$$t(n) = t\left(\sqrt{n}\right) + 1$$

$$t(n) = t(n-1) + n$$

$$t(n) = 2t(n-1) + 1$$

سوال چهارم:

الف) با توجه به کد های داده شده پیچیدگی زمانی آن ها را با نماد مناسب مشخص کنید و دلیل خود را مختصر توضیح دهید.

در صورت تفاوت بین worst case و best case آن را هم ذکر کنید.

```python
def sort_A(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr
arr = [5, 2, 8, 1, 3]
print(sort_A(arr))
```

```python
def sort_B(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr
arr = [5, 2, 8, 1, 3]
print(sort_B(arr))
```

ب) فرض کنید به الگوریتمی با پیچیدگی $O(n^2)$ ورودی با اندازه ی 10 میدهیم. اگر اندازه ی ورودی به 2n افزایش پیدا کند. تعداد عملیات ها چه تغییری میکند؟ محاسبات خود را نوشته و نتیجه گیری خود را از این مساله بیان کنید.

سوال پنجم:

کد زیر با توجه به موارد خواسته شده بهینه کنید.

```python
def has_pair(arr, target):
    for i in range(len(arr)):
        for j in range(i + 1, len(arr)):
            if arr[i] + arr[j] == target:
                return True
    return False
arr = list(map(int, input().split()))
target = int(input())
print(has_pair(arr, target))
```

بدون تغییر در خروجی. بهترین روش را از نظر پیچیدگی زمانی و حافظه ی اضافه انتخاب کرده و کدش رو بنوسید. در آخر پیچیدگی زمانی قبل و بعد از بهینه سازی
خود را بنویسید و کوتاه و مختصر توضیح دهید.
توجه: اول کدتون رو با این ورودی و خروجی ها تست کنید رو سیستم و وقتی مطمن شدید بعد روی برگه وارد کنید.

ورودی نمونه:

2 8 5 13 20

10

خروجی نمونه:

True